Diego León Acosta

→ Pregunta 0.1. Contesta con tus palabras, en una o tres oraciones. Puedes apoyarte en un ejemplo pequeño de código en la radiografía de un objeto, ¿qué representa un atributo y qué representa un método? Da un ejemplo de cada uno.

Un atributo representa el estado o las características del objeto (guarda datos).

Un método representa las acciones o comportamientos del objeto que puede realizar.

Objeto: Coche

atributo: Velocidad Método: acelerar()

→ Pregunta 0.2. ¿Qué hace el método main y por qué se declara static en los programas que usamos en clase?

El metodo main es el punto de entrado, donde se inicializa el programa.

Se declara static por la maquina Virtual, pues necesita llamarlo para iniciar el programa sin necesidad de crear un objeto de la clase primero.

→ Pregunta 0.3. ¿Que diferencia hay entre un atributo de instancia y uno static? Explica qué comparten dos objetos de la misma clase.

Un atributo de instancia, pertenece a cada objeto, por lo que, cada objeto tiene su propia copia y puede tener valores distintos).
Un atributo static pertenece a esta clase, por lo que existe una sola copia en memoria. Dos objetos de la misma clase comparten exactamente el mismo valor de atributo.

→ Pregunta 0.4. ¿Qué guarda una variable de tipo primitivo y qué guarda una de referencia? Da un ejemplo de cada una.

Primitiva: Guarda directamente el valor en la memoria
ejemplo: int edad = 20;

Referencia: Guarda la dirección de memoria (el espacio) donde se encuentra un objeto.
ejemplo. String nombre = "Ana";
Coche miCoche = new Coche();

KYMA

Diego León Acosta.

Pregunta 0.5. ¿Qué hace javac con un archivo .java y qué hace la JVM con el resultado?

Javac es el compilador, toma el código fuente .java y lo traduce a bytecode, creando un archivo .class, la JVM toma ese archivo .class y ejecuta el bytecodo. La JVM no crea archivos de resultados, solo ejecuta.

R1. ¿Qué imprime?

```
int [] datos = {7, 5, 3};
int [] alias = datos;
int i = 0;
while (i < 3) {
    System.out.println (alias[i] / 2);
    i++;
}
alias = new int[]{9, 8, 6};
System.out.println ("B" + datos[0] + alias[0]);
double mitad = (double) (datos[1] / 2);
System.out.println (mitad);
```

Salida:

3
2
1
B79
2.0

Variable / linea	i	alias [i]	Operación	Impresión en ...
While vuelta 1	0	7	7/2 = 3	3
While vuelta 2	1	5	5/2 = 2	2
While vuelta 3	2	3	3/2 = 1	1
alias = new...	—	—	alias ahora es {9, 8, 6}	—

KYMA

MPOO-2027-1 Diego León Acosta

Variable/línea	i	alias[i]	Operación	Impresión en consola.
System.out...	—	"B" + datos 0 + alias 0	→	B79
double mitad =...	—	(double)(datos [1]/2 = (double)(2)	→	2.0

R2 a) · ¿Compila, truena o funciona?

```
int modo = 1;
switch (modo){
    case 1: System.out.println("listo");
    case 2: System.out.println("FIN");
}
```

FUNCIONA

Imprime Listo y luego FIN. Funciona gracias a que Java, si no se coloca un breack en un caso, ocurre un fall-through (caída libre): la ejecución continúa hacia el siguiente caso hasta encontrar un break o el final del switch.

R2 b) ¿Compila, truena o funciona?

```
String etiqueta = null;
System.out.println(etiqueta.trim());
```

Compila pero truena.
Compila porque etiqueta es un string, pero truena en ejecución lanzando una NullPointerException. Debido a que etiqueta es null (no apunta a ningún objeto) y estamos intentando llamar al método .trim() sobre la nada.

R2 c) · ¿Compila, truena o funciona?

~~String etiqueta = null;~~
~~System.out.println(etiqueta.trim());~~
```
{int nivel = 4; }
System.out.println(nivel);
```

NO COMPILA.

La variable nivel se declara dentro de un bloque de llaves {}, por lo que su alcance (scope) es local a ese bloque. Al intentar usar en el println fuera del bloque el compilador da error porque la variable no existe ahí.

KYMA

R3 · Modelado : base, reto y justificación

Cada cargador fija su capacidad al fabricarse y nace vacío. Cuenta ranuras ocupadas y cargadores fabricados.
Escribe colocar() y una consulta de ocupadas. Supón que solo se llama colocar cuando hay espacio. No uses arreglos.

```
public class Cargador{
    private final int capacidad; // Constante final
    private int ocupadas;        // atributo de instancia
    private static int fabricados = 0 //Atributo de clase.

    public Cargador(int capacidad) {
        this.capacidad = capacidad;
        this.ocupadas = 0;
        fabricados ++;
    }

    public void colocar() {
        ocupadas ++;
    }
    public int consultarOcupadas(){
        return ocupadas;
    }
}
```

R4 · Código con repetición

El arreglo intentos contiene exactamente tres códigos enteros. Devuelve el número del primer intento que coincide con clave (1, 2 o 3)

```
static int intentoValido(int[] intentos, int clave)...
```

Respuesta:

```
static int intentoValido(int[] intentos, int clave) {
    int i = 0;
    do {
        if (intentos[i] == clave) {
            return i + 1;
        }
        i++;
    } while (i < 3);

    return 0;
}
```

KYMA

	intentos	
i = 0	[0](13) ! = 42	i pasa a 1
i = 1	[1](42) == 42	i + 1 = 2
i = 0	[0](42) == 42	Devuelve i+1 = 1

↳ ninguno coincide. Termina el do-while Devuelve 0.

R5: Codigo con selección
Devuelve la acción del módulo con switch sobre Modo.

Regla o valor recibido	Texto devuelto.
APAGADO	"Detener"
MEDIR	"Leer"
CALIBRAR	"Ajustar"
PRUEBA	"Simular"

```
static Sting accion(Modo modo, int bateria){
    if (bateria < 10) {
        return "Cargar";
    }
    if (modo == null){
        return "sin modo";
    }
    switch (modo){
        case APAGADO: return "Detener";
        case MEDIR: return "Leer";
        case CALIBRAR: return "Ajustar";
        case PRUEBA: return "Simular";
        default: return "sin modo";
    }
}
```

Si bateria < 10 → CARGAR

Si bateria >= 10 y modo es null → "SIN MODO"

Si el modo es APAGADO → DETENER

Si el modo es MEDIR → Leer

Si el modo es CALIBRAR → Ajustar

Si el modo es PRUEBA → Simular

KYMA